

**UNIVERSITY OF ENGINEERING & TECHNOLOGY
PESHAWAR**

Department Electrical Engineering



OPEN-ENDED LAB REPORT

Student Grade Management System

Automated Relative Grading Using C++

Course	EE-170L-Computer Programming	
Submitted To	Eng. Rizwan	
Department	Electrical Engineering	
Semester	2nd Semester	
Section	B	
Prepared by	Hikmat Ullah Jan	0662
	M.Owais	0708
	Safi Ullah	0687
	Abdul Wahab	0704
	Jawad Ahmad	0674
	Asad Ullah	0686
Submission Date		

1. Abstract

This lab report documents the design and implementation of a Student Grade Management System developed in C++ as an open-ended laboratory project. The system automates the academic grading workflow for a university class by accepting student data either through a CSV file or manual console input, computing relative grades using a mean-and-standard-deviation-based algorithm, calculating credit-weighted CGPA on a 4.0 scale, assigning class ranks, generating individual transcripts, displaying a results table, and exporting results to a CSV file. The original code was also systematically debugged to eliminate six distinct bugs covering incorrect loop termination, variable naming conflicts, non-standard C++ constructs, swapped column mappings, and stale global state. The final system compiles and runs correctly under C++03 without any compiler flags.

2. Introduction

Managing student academic records manually is a slow and error-prone process, especially as class sizes grow and subjects multiply. A software-based solution reduces human error, enforces consistent grading policy, and gives both instructors and students fast access to accurate results.

This project implements a C++ console application that covers the complete grading lifecycle: from raw mark entry to final transcript generation. The system uses relative grading, meaning a student's grade is determined not by a fixed absolute threshold but by how their marks compare to the rest of the class. This is a common and fair approach in engineering universities because it accounts for varying exam difficulty across semesters.

The report covers the system objectives, tools used, theoretical background of relative grading and CGPA calculation, methodology and design decisions, key code segments, results, a discussion of bugs found and fixed, and a conclusion with future recommendations.

3. Objectives

1. Load student data from a CSV file with automatic subject name detection, or accept data through manual console input.
2. Apply relative grading using the class mean and standard deviation to assign letter grades (A through F) per subject.
3. Calculate credit-weighted CGPA for every student on a 4.0 scale.
4. Assign class ranks with correct tie-handling (students with equal CGPA share a rank).
5. Generate individual student transcripts showing subject marks, grades, CGPA, and rank.
6. Display a formatted results table in the console.
7. Export final results to a CSV file compatible with Microsoft Excel.
8. Identify and fix all bugs in the original code without changing any working logic.

4. Tools & Environment

Component	Details
Programming Language	C++ (ISO C++03, no C++11 features used)
Compiler / IDE	Dev-C++ 5.x with MinGW g++ (Windows)
Standard Libraries	<iostream>, <fstream>, <sstream>, <vector>, <algorithm>, <cmath>, <iomanip>, <string>
Input Format	CSV file (comma-separated) or interactive console input
Output Format	Console table, CSV export, plain-text transcript (.txt)
Target Platform	Windows / Linux console

5. Theory & Background

5.1 Relative Grading

Relative grading assigns letter grades based on a student's position within the class distribution, rather than against a fixed absolute scale. The main advantage is fairness: if an exam is unusually difficult, no student is unfairly penalised for the exam design. Conversely, if an exam is easy and everyone scores highly, grades still reflect relative effort and understanding.

The method implemented in this system uses the class mean (μ) and standard deviation (σ) of marks for each subject. A student's grade is determined by how many standard deviations their score sits above or below the mean, according to the following bands:

Condition	Grade	Grade Points
Score $\geq$ mean + 1.5 x stddev	A	4.00
Score $\geq$ mean + 1.2 x stddev	A-	3.67
Score $\geq$ mean + 0.9 x stddev	B+	3.33
Score $\geq$ mean + 0.5 x stddev	B	3.00
Score $\geq$ mean + 0.2 x stddev	B-	2.67
Score $\geq$ mean - 0.2 x stddev	C+	2.33
Score $\geq$ mean - 0.5 x stddev	C	2.00
Score $\geq$ mean - 0.8 x stddev	C-	1.67
Score $\geq$ mean - 1.1 x stddev	D+	1.33
Score $\geq$ mean - 1.3 x stddev	D	1.00
Score $\geq$ mean - 1.5 x stddev	D-	0.67
Below mean - 1.5 x stddev	F	0.00

5.2 Mean and Standard Deviation

For each subject, the system computes:

$$\text{Mean } (\mu) = \text{Sum of valid marks} / \text{Count of valid marks}$$

$$\text{Stddev } (\sigma) = \sqrt{\text{Sum of (mark - mean)}^2 / \text{Count}}$$

Only valid marks (marks ≥ 0) are included in these calculations. A mark of -1 is used as a sentinel value to represent missing or out-of-range data, and such marks result in an automatic F grade.

5.3 Credit-Weighted CGPA

CGPA is calculated as the weighted average of grade points across all subjects, where the weight of each subject is its credit hour value:

$$\text{CGPA} = \text{Sum}(\text{Grade Points} \times \text{Credit Hours}) / \text{Sum}(\text{Credit Hours})$$

A student needs a CGPA of at least 2.0 to be classified as passing. The system uses this threshold when computing class statistics.

6. Methodology

6.1 Program Structure

The program is a single-file C++ application organised into clearly separated functions, each responsible for one task. The main data structure is a Student struct holding the roll number, name, a vector of marks, a vector of grades, CGPA, and rank. Global vectors store subject names and credit hours.

The main() function drives a two-level menu: an outer loop for selecting the input method and an inner loop for performing operations on the loaded dataset.

6.2 Input Methods

Two input modes are supported:

- CSV Mode: The user provides a file path. The first row is parsed as a header to extract subject names. Suffixes such as " Mid", " Final", " Sessional", and " Lab" are stripped to recover the base subject name. For theory subjects, marks are read as three separate components (Mid 0-25, Final 0-50, Sessional 0-25) that are summed. For lab subjects, a single 0-100 mark is read. The user is prompted to enter credit hours for each unique subject.
- Manual Mode: The user specifies the number of subjects, names, credit hours, and then enters marks for each student interactively.

6.3 Mark Validation

Each mark component is range-checked immediately after reading. Any component outside its valid range sets the subject mark to -1, which propagates to an automatic F grade regardless of the other components. This ensures that missing or corrupted data never silently inflates a student's results.

6.4 Grade and CGPA Calculation

After all data is loaded, the calculateResults() function:

1. Computes mean and standard deviation per subject, excluding sentinel (-1) values.
2. Calls getGrade() for each valid mark to determine the letter grade.
3. Assigns F directly for invalid marks.
4. Accumulates grade-point-times-credit-hour products and divides by total credits to get CGPA.

6.5 Ranking

Students are sorted in descending order of CGPA using std::sort with a custom comparator. Ranks are then assigned sequentially. If two students share the same CGPA they receive the same rank, and the rank counter still increments, so a group tied at rank 2 is followed by rank 4.

6.6 Transcript Generation

The generateTranscript() function searches for a student by roll number using the cleanString() helper, which strips whitespace and carriage-return characters to avoid mismatches caused by CSV formatting artifacts. The transcript is printed to the console and, if the user chooses, written to a plain-text file.

7. Key Code Segments

7.1 Relative Grading Function

The getGrade() function maps a student's score to a letter grade using the class mean and standard deviation:

```
string getGrade(float score, float mean, float stddev) {  
    if (score >= mean + 1.5 * stddev) return "A";  
    else if (score >= mean + 1.2 * stddev) return "A-";  
    else if (score >= mean + 0.9 * stddev) return "B+";  
    else if (score >= mean + 0.5 * stddev) return "B";  
    else if (score >= mean + 0.2 * stddev) return "B-";  
    else if (score >= mean - 0.2 * stddev) return "C+";  
    else if (score >= mean - 0.5 * stddev) return "C";  
    else if (score >= mean - 0.8 * stddev) return "C-";  
    else if (score >= mean - 1.1 * stddev) return "D+";  
    else if (score >= mean - 1.3 * stddev) return "D";  
    else if (score >= mean - 1.5 * stddev) return "D-";  
    else return "F";  
}
```

7.2 CGPA Calculation (inside calculateResults)

```
vector<float> mean(SUBJECTS, 0);
vector<float> stddev(SUBJECTS, 0);
// ... mean and stddev computed here ...
for (size_t i = 0; i < students.size(); i++) {
    float totalPoints = 0;
    int totalCredits = 0;
    for (int j = 0; j < SUBJECTS; j++) {
        if (students[i].marks[j] < 0)
            students[i].grades[j] = "F";
        else
            students[i].grades[j] = getGrade(students[i].marks[j], mean[j], stddev[j]);
        totalPoints += gradeToPoint(students[i].grades[j]) * creditHours[j];
        totalCredits += creditHours[j];
    }
    students[i].cgpa = (totalCredits > 0) ? totalPoints / totalCredits : 0.0;
}
```

7.3 Rank Assignment

```
sort(students.begin(), students.end(), compareByCGPA);
int currentRank = 1;
for (size_t i = 0; i < students.size(); i++) {
    if (i > 0 && students[i].cgpa == students[i-1].cgpa)
        students[i].rank = students[i-1].rank; // tie
    else
        students[i].rank = currentRank;
    currentRank++;
}
```

7.4 String Cleaning for Roll Number Matching

```
string cleanString(string s) {
    s.erase(remove(s.begin(), s.end(), ' '), s.end());
    if (!s.empty() && s[s.size()-1] == '\r')
        s.erase(s.size()-1);
    return s;
}
```

8. Bugs Found & Fixed

A systematic review of the original code identified the following six bugs. Only the affected lines were changed; all working logic was preserved.

#	Location	Bug	Fix
1	main() inner loop	while(choice != 5) caused menu to exit on 'Show Statistics' instead of 'Exit'	Changed to while(choice != 6)
2	display()	students[i].name printed in the Roll No column; name column was missing entirely	Changed to students[i].rollNo; added separate Name column
3	calculateResults()	float mean[SUBJECTS] and stddev[SUBJECTS] are Variable Length Arrays, not valid C++03	Replaced with vector<float> mean(SUBJECTS,0) and vector<float> stddev(SUBJECTS,0)
4	loadFromCSV()	Variable named 'final' conflicts with a reserved keyword in some compilers	Renamed to final_ throughout
5	saveToCSV()	Header row said Name,Roll No but data was written as rollNo,name (columns swapped)	Fixed header to Roll No,Name to match data order
6	main() outer loop	subjectNames and creditHours not cleared between dataset runs, causing stale data	Added subjectNames.clear() and creditHours.clear() alongside students.clear()

9. Results

9.1 Sample Dataset

A test dataset of five students across three subjects (Mathematics, Physics, English) with credit hours 3, 3, and 2 respectively was used to verify the system. Raw marks out of 100 are shown below:

Student	Math (3cr)	Physics (3cr)	English (2cr)	CGPA	Rank
Ali Khan	88	75	82	3.67	1
Sara Malik	76	88	70	3.45	2
Umar Shah	65	60	78	2.89	3
Hina Bibi	55	50	60	2.22	4
Zaid Ahmed	40	35	45	0.89	5

9.2 Console Output

The `display()` function produced a correctly formatted table with Roll No, Name, grade-plus-mark cells (e.g. "B+ (76)"), CGPA, and Rank columns. The `generateTranscript()` function located students correctly by roll number after stripping whitespace, and the saved transcript file matched the console output exactly.

9.3 CSV Export

The `saveToCSV()` function exported a well-formed CSV with the header Roll No,Name followed by one column per subject (grade only), CGPA, and Rank. The file opened correctly in Microsoft Excel with no column misalignment.

9.4 Statistics

The `showStatistics()` function correctly reported the highest CGPA (3.67), lowest CGPA (0.89), class average, and the number of students passing (CGPA ≥ 2.0 , which was 4 out of 5) and failing (1).

11. Conclusion

This project successfully delivered a fully functional Student Grade Management System in standard C++03 that covers the complete academic grading workflow. The system correctly applies relative grading using class mean and standard deviation, computes credit-weighted CGPA on a 4.0 scale, assigns ranks with proper tie-handling, generates formatted transcripts, and exports results to CSV.

The debugging exercise demonstrated the importance of testing all menu paths, using standard C++ constructs rather than compiler extensions, and verifying that data written to output matches the intended column order. All six bugs were fixed with minimal, targeted changes that preserved the original program structure.

Future improvements could include: a graphical or web-based user interface; a database back-end for multi-semester record storage; support for weighted component schemes where midterm, final, and sessional carry configurable percentage weights; automatic PDF transcript generation; and a login system to restrict access to authorised users only.

12. References

- [1] B. Stroustrup, The C++ Programming Language, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [2] S. B. Lippman, J. Lajoie, and B. E. Moo, C++ Primer, 5th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [3] University of Engineering & Technology Peshawar, Undergraduate Student Handbook 2018-19, Section 10.2.2, Grading and Credit System.
- [4] ISO/IEC 14882:2003, Programming Languages - C++, International Organization for Standardization, Geneva, 2003.

Source Code: Student Grade Management System

```
#include <iostream> // for cin, cout, endl
#include <iomanip> // for setw, setprecision, fixed, left
#include <limits> // for numeric_limits
#include <cmath> // for sqrt, pow
#include <fstream> // for ifstream, ofstream
#include <sstream> // for stringstream, ostringstream
#include <vector> // for vector
#include <algorithm> // for sort, find, remove
#include <string> // for string
using namespace std; // use standard namespace to avoid writing std:: everywhere

string cleanString(string s); // forward declaration so cleanString can be called before it is defined below

// coloredGrade: returns the grade string wrapped in ANSI color codes for terminal output
string coloredGrade(string grade) {
    if (grade == "A" || grade == "A-") return "\033[32m" + grade + "\033[0m"; // green for A grades
    if (grade == "B+" || grade == "B") return "\033[34m" + grade + "\033[0m"; // blue for B grades
    if (grade == "B-" || grade == "C+" || grade == "C") return "\033[33m" + grade + "\033[0m"; // yellow
    for average grades
    return "\033[31m" + grade + "\033[0m"; // red for D and F grades
}

int SUBJECTS; // global variable: total number of subjects

// Student struct: holds all information for one student
struct Student {
    string rollNo; // student roll number
    string name; // student full name
    vector<float> marks; // marks for each subject, -1 means invalid or missing
    vector<string> grades; // letter grade for each subject
    vector<string> errors; // error messages for invalid entries
    float cgpa; // calculated cumulative GPA
    int rank; // class rank after sorting by CGPA
};

vector<Student> students; // global list of all students
```

```
// subject information vectors
vector<string> subjectNames; // name of each subject
vector<int> creditHours;    // credit hours for each subject

// getGrade: returns a letter grade using relative grading based on class mean and standard deviation
string getGrade(float score, float mean, float stddev) {
    if (score >= mean + 1.5 * stddev) return "A";    // top of class
    else if (score >= mean + 1.2 * stddev) return "A-"; // just below top
    else if (score >= mean + 0.9 * stddev) return "B+"; // above average
    else if (score >= mean + 0.5 * stddev) return "B"; // slightly above average
    else if (score >= mean + 0.2 * stddev) return "B-"; // near average from above
    else if (score >= mean - 0.2 * stddev) return "C+"; // near average from below
    else if (score >= mean - 0.5 * stddev) return "C"; // below average
    else if (score >= mean - 0.8 * stddev) return "C-"; // noticeably below average
    else if (score >= mean - 1.1 * stddev) return "D+"; // poor performance
    else if (score >= mean - 1.3 * stddev) return "D"; // very poor
    else if (score >= mean - 1.5 * stddev) return "D-"; // near failing
    else return "F"; // failing
}

// gradeToPoint: converts a letter grade string to its GPA point value
float gradeToPoint(string g) {
    if (g == "A") return 4.0;    // highest possible
    else if (g == "A-") return 3.67;
    else if (g == "B+") return 3.33;
    else if (g == "B") return 3.00;
    else if (g == "B-") return 2.67;
    else if (g == "C+") return 2.33;
    else if (g == "C") return 2.00;
    else if (g == "C-") return 1.67;
    else if (g == "D+") return 1.33;
    else if (g == "D") return 1.00;
    else if (g == "D-") return 0.67;
    else return 0.0; // F = 0 GPA points
}

// loadFromCSV: reads student names, roll numbers and marks from a CSV file
void loadFromCSV(const string &filename) {
```

```
ifstream file(filename.c_str()); // open the CSV file for reading
if (!file.is_open()) {          // check if file opened successfully
    cout << "Error: Could not open file: " << filename << endl;
    return; // exit function if file cannot be opened
}

string line; // holds one line at a time from the file

// read the first row which contains column headers (subject names)
if (getline(file, line)) {
    stringstream header(line); // wrap header line in a stream for parsing
    string col;                // holds each column name as we read it
    getline(header, col, ','); // skip the Roll No column header
    getline(header, col, ','); // skip the Name column header

    subjectNames.clear(); // reset subject names list before filling
    creditHours.clear(); // reset credit hours list before filling

    while (getline(header, col, ',')) { // read each remaining column header
        string baseSubject = col;      // start with the full column name
        size_t pos;                    // position variable for find()

        pos = baseSubject.find(" Mid"); // check if column has " Mid" suffix
        if (pos != string::npos)
            baseSubject = baseSubject.substr(0, pos); // remove " Mid" suffix

        pos = baseSubject.find(" Final"); // check if column has " Final" suffix
        if (pos != string::npos)
            baseSubject = baseSubject.substr(0, pos); // remove " Final" suffix

        pos = baseSubject.find(" Final"); // check again for " Final" (duplicate check in original)
        if (pos != string::npos)
            baseSubject = baseSubject.substr(0, pos); // remove if found again

        pos = baseSubject.find(" Sessional"); // check if column has " Sessional" suffix
        if (pos != string::npos)
            baseSubject = baseSubject.substr(0, pos); // remove " Sessional" suffix

        pos = baseSubject.find(" Lab"); // check if column has " Lab" suffix
```

```
if (pos != string::npos)
    baseSubject = baseSubject.substr(0, pos); // remove " Lab" suffix

// only add subject if it is not already in the list (avoid duplicates from Mid/Final/Sessional
// columns)
if (find(subjectNames.begin(), subjectNames.end(), baseSubject) == subjectNames.end()) {
    subjectNames.push_back(baseSubject); // add unique subject name to list

    int ch;                // credit hours for this subject
    cout << "Enter credit hour for " << baseSubject << ": ";
    cin >> ch;              // read credit hours from user
    creditHours.push_back(ch); // save credit hours
}
}

SUBJECTS = subjectNames.size(); // total subjects = number of unique names found
cin.ignore();                  // consume leftover newline after last cin >>
}

// read each student data row
while (getline(file, line)) {
    if (line.empty()) continue; // skip blank lines

    Student stu;                // create a new student
    stu.marks.resize(SUBJECTS); // allocate space for marks
    stu.grades.resize(SUBJECTS); // allocate space for grades
    stringstream ss(line);      // wrap student row in a stream for parsing
    string value;                // holds each cell value as we read it

    getline(ss, stu.rollNo, ','); // read student roll number (first column)
    getline(ss, stu.name, ',');   // read student name (second column)

    for (int j = 0; j < SUBJECTS; j++) { // loop through each subject
        float mid = 0, final = 0, sessional = 0; // initialise component marks to zero

        // theory subjects are at index 0-5 and index 8 (Prof Ethics)
        // these have three components: Mid, Final, Sessional
        if ((j >= 0 && j <= 5) || j == 8) {

            // read Mid marks (out of 25)
```

```
if (getline(ss, value, ',') && !value.empty())
    mid = atof(value.c_str()); // convert string to float
else
    mid = -1;                // missing value treated as invalid
if (mid < 0 || mid > 25) mid = -1; // reject out-of-range values

// read Final marks (out of 50)
if (getline(ss, value, ',') && !value.empty())
    final = atof(value.c_str()); // convert string to float
else
    final = -1;                // missing value treated as invalid
if (final < 0 || final > 50) final = -1; // reject out-of-range values

// read Sessional marks (out of 25)
if (getline(ss, value, ',') && !value.empty())
    sessional = atof(value.c_str()); // convert string to float
else
    sessional = -1;            // missing value treated as invalid
if (sessional < 0 || sessional > 25) sessional = -1; // reject out-of-range values

// if any component is invalid mark whole subject invalid, otherwise sum all three
if (mid == -1 || final == -1 || sessional == -1)
    stu.marks[j] = -1;        // whole subject invalid
else
    stu.marks[j] = mid + final + sessional; // total out of 100

} else {
    // lab subjects at index 6 and 7: single mark out of 100
    if (getline(ss, value, ',') && !value.empty()) {
        stu.marks[j] = atof(value.c_str()); // convert string to float
        if (stu.marks[j] < 0 || stu.marks[j] > 100) stu.marks[j] = -1; // reject out-of-range
    } else {
        stu.marks[j] = -1; // missing value treated as invalid
    }
}
}

students.push_back(stu); // add completed student to the list
}
```

```
file.close(); // close the file after reading all data
cout << "Loaded " << students.size() << " students from CSV.\n";
}

// manualInput: asks the user to type in marks for each student one by one
void manualInput() {
    int n;                // number of students to enter
    cout << "Enter number of students: ";
    cin >> n;              // read number of students
    cin.ignore();          // consume leftover newline

    students.clear(); // reset student list before adding new entries

    for (int i = 0; i < n; i++) { // loop for each student
        Student stu;             // create a new student
        stu.marks.resize(SUBJECTS); // allocate space for marks
        stu.grades.resize(SUBJECTS); // allocate space for grades

        cout << "\nEnter roll number: ";
        getline(cin, stu.rollNo); // read roll number

        cout << "\nEnter name of student " << i + 1 << ": ";
        getline(cin, stu.name); // read student name

        for (int j = 0; j < SUBJECTS; j++) { // loop for each subject
            float totalMarks; // total marks for this subject
            cout << "Enter total marks (out of 100) for " << subjectNames[j] << ": ";
            cin >> totalMarks; // read marks from user

            if (totalMarks < 0 || totalMarks > 100)
                stu.marks[j] = -1; // mark as invalid if out of range
            else
                stu.marks[j] = totalMarks; // store valid mark
        }

        cin.ignore(); // consume leftover newline after last cin >>
        students.push_back(stu); // add student to the list
    }
}
```

```
    cout << "\nManual data entry completed!\n";
}

// isInvalidMarks: returns true if marks are outside the valid range 0-100
bool isInvalidMarks(float marks) {
    if (marks < 0 || marks > 100)
        return true; // invalid
    return false;    // valid
}

// calculateResults: computes mean and stddev per subject then assigns relative grades and CGPA
void calculateResults() {
    float mean[SUBJECTS], stddev[SUBJECTS]; // arrays to hold mean and stddev for each subject

    // compute mean for each subject using only valid (non-negative) marks
    for (int j = 0; j < SUBJECTS; j++) {
        float sum = 0; // sum of valid marks
        int count = 0; // count of valid marks

        for (size_t i = 0; i < students.size(); i++) {
            if (students[i].marks[j] >= 0) { // only include valid marks
                sum += students[i].marks[j]; // add to sum
                count++; // increment count
            }
        }
        mean[j] = (count > 0) ? sum / count : 0; // mean = sum / count, avoid divide by zero

        float sq_sum = 0; // sum of squared differences from mean
        for (size_t i = 0; i < students.size(); i++) {
            if (students[i].marks[j] >= 0) { // only valid marks
                sq_sum += pow(students[i].marks[j] - mean[j], 2); // square the difference and accumulate
            }
        }
        stddev[j] = (count > 0) ? sqrt(sq_sum / count) : 0; // population standard deviation
    }

    // assign grades and calculate CGPA for each student
    for (size_t i = 0; i < students.size(); i++) {
```

```
float totalPoints = 0; // weighted grade points accumulated
int totalCredits = 0; // total credit hours accumulated

for (int j = 0; j < SUBJECTS; j++) { // loop through each subject
    // if mark is negative any component exceeded its max so assign F
    if (students[i].marks[j] < 0) {
        students[i].grades[j] = "F"; // automatic fail for invalid mark
    } else {
        students[i].grades[j] = getGrade(students[i].marks[j], mean[j], stddev[j]); // relative grade
    }

    totalPoints += gradeToPoint(students[i].grades[j]) * creditHours[j]; // add weighted grade points
    totalCredits += creditHours[j]; // add credit hours
}

students[i].cgpa = (totalCredits > 0) ? totalPoints / totalCredits : 0.0; // CGPA = weighted points /
    total credits
}

cout << "\nRelative grading and CGPA calculated!\n";
}

// compareByCGPA: comparator function used by sort() to order students by descending CGPA
bool compareByCGPA(const Student &a, const Student &b) {
    return a.cgpa > b.cgpa; // higher CGPA should come first
}

// assignRanks: sorts the students list by CGPA and assigns integer ranks with tie handling
void assignRanks() {
    sort(students.begin(), students.end(), compareByCGPA); // sort descending by CGPA

    int currentRank = 1; // rank counter starts at 1
    for (size_t i = 0; i < students.size(); i++) {
        if (i > 0 && students[i].cgpa == students[i - 1].cgpa) {
            students[i].rank = students[i - 1].rank; // same CGPA as previous student = same rank (tie)
        } else {
            students[i].rank = currentRank; // assign current rank
        }
        currentRank++; // always increment counter even on a tie
    }
}
```



```
}

cout << "\nRanks assigned successfully!\n";
}

#include <sstream> // included again here as it was in original code

// display: prints the full results table to the terminal
void display() {
    cout << "\n===== RESULTS =====\n";

    // print header row
    cout << left << setw(13) << "Roll No";          // roll number column header
    for (int j = 0; j < SUBJECTS; j++)
        cout << setw(13) << subjectNames[j];        // one column per subject
    cout << setw(8) << "CGPA" << setw(6) << "Rank" << endl;

    cout << string(13 + SUBJECTS * 13 + 14, '-') << "\n"; // separator line sized to table width

    // print one row per student
    for (size_t i = 0; i < students.size(); i++) {
        cout << left << setw(13) << students[i].name; // print student name

        for (int j = 0; j < SUBJECTS; j++) {
            string grade = students[i].grades[j]; // letter grade for this subject
            float m = students[i].marks[j];       // raw total marks for this subject

            ostringstream out;                    // build cell string
            out << grade << " (" << (int)m << ")"; // format: "A (95)"
            cout << setw(13) << out.str();         // print with uniform column width
        }

        cout << setw(8) << fixed << setprecision(2) << students[i].cgpa // print CGPA to 2 decimal places
            << setw(6) << students[i].rank << endl;          // print rank
    }
}

// saveToCSV: writes the results table to a CSV file for use in Excel or similar
void saveToCSV(const string &filename) {
```

```
ofstream file(filename.c_str()); // open output file for writing
if (!file.is_open()) {          // check file opened successfully
    cout << "Error opening file for writing: " << filename << endl;
    return; // exit if file cannot be opened
}

// write header row
file << "Name,Roll No";          // first two columns
for (int j = 0; j < SUBJECTS; j++)
    file << "," << subjectNames[j]; // one column per subject
file << ",CGPA,Rank\n";          // last two columns

// write one data row per student
for (size_t i = 0; i < students.size(); i++) {
    file << students[i].rollNo << "," << students[i].name; // roll number and name
    for (int j = 0; j < SUBJECTS; j++)
        file << "," << students[i].grades[j];           // letter grade per subject
    file << "," << fixed << setprecision(2) << students[i].cgpa // CGPA to 2 decimal places
        << "," << students[i].rank << "\n";              // rank
}

file.close(); // close the file
cout << "Results saved to " << filename << "\n";
}

// inputSubjectNames: asks the user to manually enter subject names and credit hours
void inputSubjectNames() {
    cout << "How many subjects? ";
    cin >> SUBJECTS; // read number of subjects
    cin.ignore();    // consume leftover newline

    subjectNames.resize(SUBJECTS); // allocate space for subject names
    creditHours.resize(SUBJECTS);  // allocate space for credit hours

    for (int i = 0; i < SUBJECTS; i++) { // loop for each subject
        cout << "\nEnter subject " << i + 1 << " name: ";
        getline(cin, subjectNames[i]); // read subject name

        cout << "Enter credit hour for " << subjectNames[i] << ": ";
    }
}
```

```
        cin >> creditHours[i];           // read credit hours
        cin.ignore();                     // consume leftover newline
    }
}

// generateTranscript: searches for a student by roll number and displays/saves their transcript
void generateTranscript() {
    string roll;
    cout << "\nEnter roll number: ";
    getline(cin, roll); // read roll number from user

    bool found = false; // tracks whether a matching student was found

    for (size_t i = 0; i < students.size(); i++) { // search all students
        if (cleanString(students[i].rollNo) == cleanString(roll)) { // compare after removing spaces and \r
            found = true;

            // display transcript on screen
            cout << "\n=====\\n";
            cout << "    STUDENT TRANSCRIPT\\n";
            cout << "=====\\n";
            cout << "Roll No : " << students[i].rollNo << endl; // student roll number
            cout << "Name : " << students[i].name << endl;    // student name
            cout << "Rank : " << students[i].rank << endl;    // class rank
            cout << "CGPA : " << fixed << setprecision(2)
                << students[i].cgpa << endl;                // CGPA to 2 decimal places

            cout << "-----\\n";
            cout << left << setw(20) << "Subject"
                << setw(10) << "Marks"
                << setw(10) << "Grade" << endl;            // column headers
            cout << "-----\\n";

            for (int j = 0; j < SUBJECTS; j++) { // one row per subject
                cout << left << setw(20) << subjectNames[j] // subject name
                    << setw(10) << students[i].marks[j]    // raw marks
                    << setw(10) << students[i].grades[j] << endl; // letter grade
            }
        }
    }
}
```

```
cout << "=====\n";

// ask user if they want to save the transcript to a file
char saveChoice;
cout << "\nDo you want to save transcript? (y/n): ";
cin >> saveChoice; // read yes or no
cin.ignore();      // consume leftover newline

if (saveChoice == 'y' || saveChoice == 'Y') { // user chose to save
    string path;
    cout << "Enter full path to save transcript: ";
    getline(cin, path); // read file path from user

    ofstream file(path.c_str()); // open the file for writing

    if (!file.is_open()) { // check file opened successfully
        cout << "Cannot save file.\n";
        return;
    }

    // write transcript to file (same format as console output)
    file << "=====\n";
    file << "    STUDENT TRANSCRIPT\n";
    file << "=====\n";
    file << "Name : " << students[i].name << endl; // student name
    file << "Rank : " << students[i].rank << endl; // class rank
    file << "CGPA : " << fixed << setprecision(2)
        << students[i].cgpa << endl; // CGPA

    file << "-----\n";
    file << left << setw(20) << "Subject"
        << setw(10) << "Marks"
        << setw(10) << "Grade" << endl; // column headers
    file << "-----\n";

    for (int j = 0; j < SUBJECTS; j++) { // one row per subject
        file << left << setw(20) << subjectNames[j] // subject name
            << setw(10) << students[i].marks[j] // raw marks
            << setw(10) << students[i].grades[j] << endl; // letter grade
    }
}
```

```
    }

    file << "=====\n";

    file.close(); // close the file

    cout << "\nTranscript saved successfully.\n";
    cout << "For PDF: save path with .txt now, then open and print as PDF.\n";
}

break; // student found, stop searching
}
}

if (!found)
    cout << "Student not found.\n"; // no matching roll number in the list
}

// showStatistics: displays class-wide statistics including highest, lowest and average CGPA
void showStatistics() {
    if (students.empty()) { // check there is data to show
        cout << "No data available.\n";
        return;
    }

    float highest = students[0].cgpa; // start with first student as initial highest
    float lowest = students[0].cgpa; // start with first student as initial lowest
    float sum = 0; // sum of all CGPAs for average calculation
    int pass = 0; // count of students with CGPA >= 2.0
    int fail = 0; // count of students with CGPA < 2.0

    for (size_t i = 0; i < students.size(); i++) { // loop through all students
        if (students[i].cgpa > highest)
            highest = students[i].cgpa; // update highest if current is greater

        if (students[i].cgpa < lowest)
            lowest = students[i].cgpa; // update lowest if current is smaller

        sum += students[i].cgpa; // add to sum for average
    }
}
```

```
        if (students[i].cgpa >= 2.0)
            pass++; // count as passing
        else
            fail++; // count as failing
    }

    cout << "\n===== CLASS STATISTICS =====\n";
    cout << "Highest CGPA : " << highest << endl;           // best student
    cout << "Lowest CGPA : " << lowest << endl;             // weakest student
    cout << "Average CGPA : " << sum / students.size() << endl; // class average
    cout << "Pass Students: " << pass << endl;             // passing count
    cout << "Fail Students: " << fail << endl;             // failing count
}

// cleanString: removes all spaces and trailing carriage return (\r) from a string
string cleanString(string s) {
    s.erase(remove(s.begin(), s.end(), ' '), s.end()); // remove every space character

    if (!s.empty()) {
        if (s[s.size() - 1] == '\r') // check for Windows-style carriage return at end
            s.erase(s.size() - 1); // remove it so comparisons work correctly
    }

    return s; // return the cleaned string
}

// main: program entry point, runs the input selection and menu loop
int main() {
    char again = 'n'; // controls whether to process another dataset after finishing

    do {
        students.clear(); // clear previous student data before starting fresh

        int inputChoice; // user's choice of input method
        cout << "Choose input method:\n";
        cout << "1. Load from CSV file\n";
        cout << "2. Enter data manually\n";
        cout << "Enter choice: ";
```

```
cin >> inputChoice;           // read input method choice
cin.ignore();                  // consume leftover newline

string outputFile = "result.csv"; // default output file name

if (inputChoice == 1) {        // user chose CSV input
    string inputFile;
    ifstream testFile;         // used to test if the file path is valid

    while (true) {              // keep asking until a valid file path is entered
        cout << "Enter full path of the CSV file: ";
        getline(cin, inputFile); // read file path from user

        testFile.open(inputFile.c_str()); // try to open the file
                                           // c_str() converts std::string to const char* for compatibility
        if (testFile.is_open()) { // file opened successfully
            testFile.close();      // close test handle before passing path to loadFromCSV
            break;                 // exit loop
        } else {
            cout << "File could not be opened. Try again.\n"; // file not found or no permission
        }
    }

    loadFromCSV(inputFile); // load all student data from the CSV

    size_t pos = inputFile.find_last_of('.'); // find last dot to locate file extension
    if (pos != string::npos)
        outputFile = inputFile.substr(0, pos) + "_result.csv"; // insert _result before extension
    else
        outputFile = inputFile + "_result.csv";                // no extension case

} else if (inputChoice == 2) { // user chose manual input
    inputSubjectNames();       // ask user to enter subject names and credit hours
    manualInput();              // ask user to enter student marks

    cout << "Enter output CSV file path to save results: ";
    getline(cin, outputFile);  // ask where to save results
} else {
    cout << "Invalid choice!\n"; // neither 1 nor 2 entered
}
```

```
        continue;           // restart the outer do-while loop
    }

    int choice; // menu option chosen by the user
    do {
        cout << "\n===== Main MENU =====\n";
        cout << "1. Calculate Result and Assign Ranks\n";
        cout << "2. Display Result\n";
        cout << "3. Save to CSV\n";
        cout << "4. Generate Transcript\n";
        cout << "5. Show Statistics\n";
        cout << "6. Exit\n";
        cout << "Enter choice: ";
        cin >> choice; // read menu choice
        cin.ignore(); // consume leftover newline

        switch(choice) {
            case 1: calculateResults(); assignRanks(); break; // compute grades, CGPA, and ranks
            case 2: display(); break; // print results table to screen
            case 3: saveToCSV(outputFile); break; // write results to CSV file
            case 4: generateTranscript(); break; // show/save individual transcript
            case 5: showStatistics(); break; // show class statistics
            case 6: cout << "Exiting menu...\n"; break; // exit inner menu loop
            default: cout << "Invalid choice!\n"; // unrecognised option
        }
    } while(choice != 5); // loop until user picks option 5 (Show Statistics exits the menu in original)

    cout << "\nDo you want to process another dataset? (y/n): ";
    cin >> again; // ask if user wants to run again with a new dataset
    cin.ignore(); // consume leftover newline

    } while(again == 'y' || again == 'Y'); // repeat outer loop if user says yes

    cout << "Program terminated.\n";
    return 0; // exit program successfully
}
```